SYSTEM INTEGRATION HANDOFF

Mini-ERP: Integration of Systems 1, 2 and 3

UNIVERSITY MINI-ERP PROJECT

Catalogue · Sales / Order · Inventory

Unified reference mappings, proposed database extension, business rules, and the Sales–Inventory integration contract.

Version 0.2 — Discussion draft

22 October 2026 · Prepared for the Sales and Inventory groups

> **Purpose**
> Extend the supplied System 1 and 2 design to include System 3, while preserving each group's record ownership. This document proposes an integration; it does not claim a live Inventory service or an approved class-wide agreement.

Read this document in three layers

Layer	Meaning
Source design	Requirements retained from the combined Catalogue/Sales document and the Inventory group's deliverables.
Current application	Behavior verified from the CampusERP source; the running demonstration uses SQLite and explicit simulators.
Proposed target	The shared MySQL schema and service agreement below, subject to review and implementation.

Document map

Scope and ownership · Reconciliation decisions · Data model and relationships · Invariants and concurrency · Lifecycle and cancellation · Existing API contract · Compatibility gaps · Test and rollout plan · Proposed additive SQL.

Based on the supplied *System 1 and 2 Document*, *System12.sql*, and *Inventory Management Deliverables*. No institutional or group approval is implied.

INTEGRATION SPECIFICATION

1. Purpose, scope and ownership

The integration joins Catalogue's seller offers to Sales' accepted order items and Inventory's stock reservations. Catalogue orderability and physical stock availability are separate checks.

System	Owns / may change	Integration responsibility
1 · Catalogue	category, product, seller, seller_offer; names, prices and active statuses	Provide stable offer identities and active-status information. No stock balances are stored in Catalogue.
2 · Sales / Order	customer, sales_order, order_item, order_status_history; accepted purchase details	Submit immutable item references; coordinate reservation, payment, fulfillment and cancellation. Never alter stock balances.
3 · Inventory	Inventory balances, reservations, movements, request/event audit records	Reserve, confirm, release, expire and deduct stock atomically. Never verify payments or rewrite Sales purchases.
4 · Billing	Invoices, payment attempts and verified results	Report authoritative payment outcomes to Sales. No direct Inventory payment feed is assumed.
5 · Delivery	Shipments, dispatch/pickup evidence and delivery confirmation	Send an agreed stock-deduction event to Inventory; report verified delivery to Sales.

Initial target scope

- One seller per order; one stock record per seller offer; whole-unit quantities.
- All-or-nothing reservation across every item in an order.
- PHP product totals remain Sales snapshots. Inventory never prices an order.
- One shipment per order, verified prepayment, and no initial COD or partial fulfillment.

Outside this handoff's implementation scope

Multi-warehouse operations, purchasing, serial/batch tracking, paid cancellation, refunds and return workflows are not added to Sales. The proposed movement vocabulary retains Inventory's restock/return/adjustment concepts; it does not enable customer returns in the current Sales app.

> **Availability is advisory**
>
> A displayed quantity is not a promise. Only an authoritative, all-item reservation can authorize the next payment step.

INTEGRATION SPECIFICATION

2. Reconciliation decisions

Topic	Source difference	Proposed resolution / approval needed
Database	Combined design uses MySQL; Inventory source discusses Firebase/Supabase.	Use MySQL 8.0.16+ / InnoDB for this proposed shared schema. Confirm platform before migration. Current demo stays SQLite.
Offer identity	Combined schema uses offer_id; Inventory uses seller_id + product_id.	Use offer_id as the canonical key. The original unique seller/product pair maps to exactly one seller_offer row. Derive product_id through that row.
Request uniqueness	Inventory lists a unique request reference on each reservation; Sales sends one reference for an entire order.	Store one request ledger row per operation. Share its reference across reservation lines; use UNIQUE(request_reference, order_item_id).
Dispatch event uniqueness	One dispatch event can include several order items.	Deduplicate the event once in an inbox; allow several linked movements. Do not make the event reference globally unique per movement row.
Confirmed reservation expiry	Inventory removes expiry after payment. The Sales adapter now supports explicit NULL confirmed expiry.	Local alignment implemented and tested. Confirm the external Inventory contract and retain explicit no-expiry semantics.
Failure and deduction policies	Inventory describes release after failed payment and deduction at dispatch; Delivery also mentions pickup.	Agree the release coordinator and exactly one deduction event. Current Sales requires cancellation checks/release; it does not deduct stock.

Decision status

These are proposed reconciliations, not approved class agreements. The reference SQL and the live adapters must be reviewed together before adoption.

Deployment choice

The SQL appendix assumes one agreed MySQL database, so cross-module foreign keys can reference the original tables. If the groups use independent databases or APIs, replace those cross-module foreign keys with verified external-reference mappings. Keep Inventory’s local integrity constraints; do not create fake local copies of another group’s operational records.

INTEGRATION SPECIFICATION

3. Unified entity model

Entity	Key / relationship	Purpose
seller_offer existing	offer_id PK; UNIQUE(seller_id, product_id)	Catalogue's authoritative seller/product offer and current price.
sales_order / order_item existing	order_id / order_item_id; one seller per order	Accepted purchase, quantities, names and prices. Existing schema remains unchanged.
inventory proposed	inventory_id PK; offer_id UNIQUE; composite offer/seller FK	Physical on-hand, reserved quantity, derived available quantity and reorder threshold.
stock_reservation proposed	reservation_id string PK; order_item_id UNIQUE	One hold for one accepted order item, linked to its inventory and originating reserve request.
stock_movement proposed	movement_id PK; sale reservation link unique	Append-only physical balance changes; no movement for reservation/release alone.
inventory_request proposed	request_reference PK; body hash and saved result	Operation-level idempotency for reserve, confirm and release.
inventory_order_control proposed	order_id PK; reservation_blocked flag	Order-level release/cancellation tombstone; blocks a delayed reserve.
inventory_event_inbox / outbox proposed	unique source / outgoing event reference	Deduplicate inbound events and durably deliver Inventory notifications after commit.

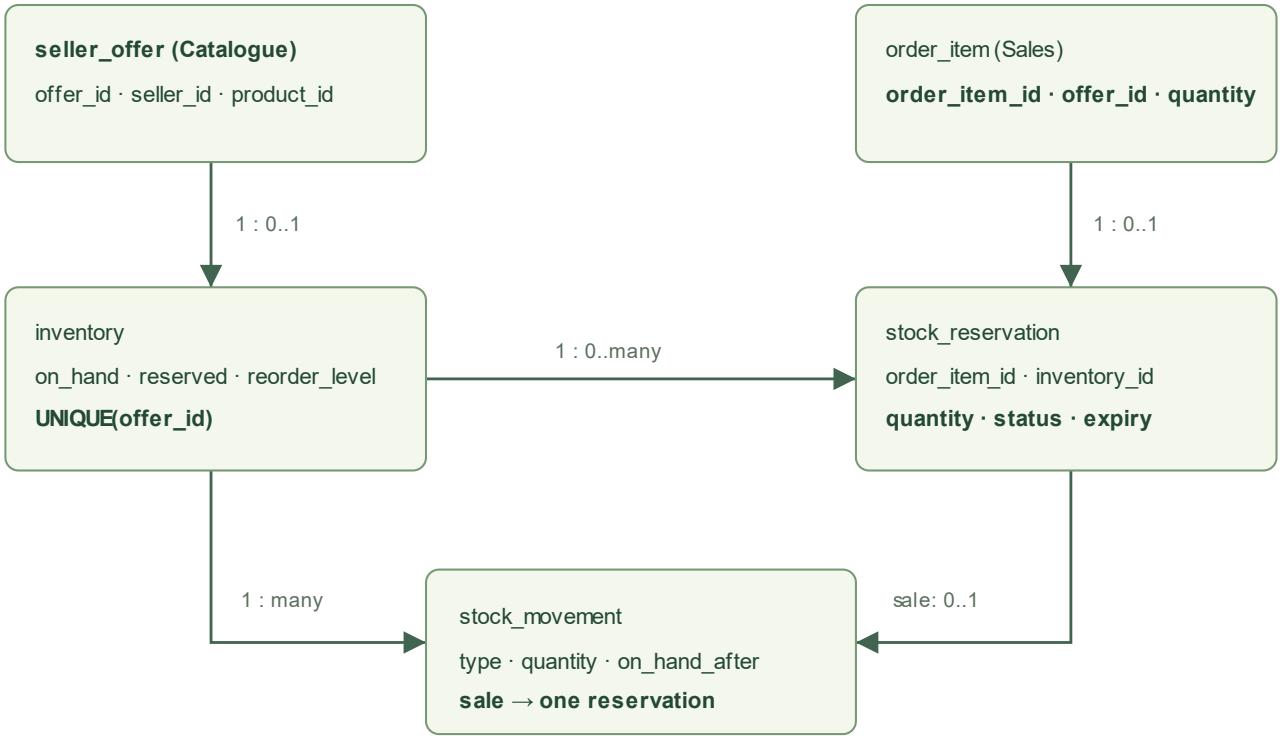
Reference validation beyond foreign keys

The original order_item table does not expose a unique composite key for all order/offer/seller/item fields. To avoid altering another group's tables, the proposed reservation has separate foreign keys. Inventory must additionally verify that the item belongs to the supplied order, offer and seller, and that the reserved quantity equals the accepted Sales quantity. Foreign keys alone do not prove that full mapping.

> **Preserve the purchase record**
>
> Do not add mutable current-price columns to Inventory. Read accepted names/prices from Sales only when needed for display; stock decisions use identities and quantities.

INTEGRATION SPECIFICATION

4. Relationships and separate states



Core relationships. Request, control and event tables support processing and audit; they do not replace the owning systems' records.

Reservation status mapping

Inventory source	Proposed stored status	Meaning / allowed next state
Active	active	Unpaid hold with expiry → confirmed, released or expired.
Confirmed	confirmed	Payment verified; expiry removed → fulfilled or released under an approved policy.
Fulfilled	fulfilled	Deduction event applied; final.
Released	released	Hold removed without physical deduction; final.
Expired	expired	Unpaid hold expired; final.

Sales order states remain exactly: `draft`, `pending_stock`, `awaiting_payment`, `confirmed`, `fulfilling`, `completed`, `cancellation_pending`, `cancelled`. A reservation state never substitutes for an order or payment state.

INTEGRATION SPECIFICATION

5. Stock invariants and concurrency

available = quantity_on_hand − quantity_reserved

Rule	Required enforcement
Nonnegative balances	on_hand ≥ 0; reserved ≥ 0; reserved ≤ on_hand; whole units only.
Available and low stock	Compute availability from balances. Low stock means available ≤ reorder_level; out of stock means available = 0.
Reservation accounting	Reserved balance equals the sum of active and confirmed reservation quantities. Periodically reconcile; do not overwrite mismatches silently.
Physical movement audit	Every change to on_hand and its movement record commit together. Movements are append-only; correct mistakes with a new movement.
All-item atomicity	Validate every order item; if one fails, create no holds and change no reserved balance. Return all known short/inactive item details.
Retention and authorization	Do not delete stock/reservations/movements used by purchases. Only Inventory's controlled service writes its balances and audit tables.

Single lock order for every mutating operation

1. Lock the Inventory order-control row for the affected order, creating it safely if absent.
2. Lock affected inventory rows in ascending inventory_id order.
3. Lock affected reservation rows in a consistent item order.
4. Apply validation, balances, reservation states, audit/result and outbox changes; commit.

Reserve, confirm, release, expiry and fulfillment all follow this order. An expiry worker first finds candidate order IDs without holding row locks, then processes each order using the same locking order. This avoids a release-versus-reserve race and reduces deadlocks. Deadlocks still require bounded retry with the original reference.

Balance example

Action	On hand	Reserved	Available
Restock 10 units	10	0	10
Reserve 3 units	10	3	7
Release those 3	10	0	10
Reserve/confirm 3, then fulfill	7	0	7

Reservation, confirmation, release and expiry do not change physical on-hand. Fulfillment changes both on-hand and reserved and records a sale movement. Never keep a transaction open while waiting for another service.

INTEGRATION SPECIFICATION

6. Normal order-to-stock workflow

Step	Sales responsibility	Inventory responsibility
1 · Draft / review	Validate customer, one seller, active offers and recipient. Calculate PHP totals and require accepted current prices.	No reservation is implied by viewing stock or building a draft.
2 · Submit	Atomically save immutable item/name/price snapshots, pending_stock, status history and reserve request.	Receive the complete accepted order item set, with stable request and item references.
3 · Reserve	Remain pending_stock on pending/rejected results. Move to awaiting_payment only after every item is reserved.	Lock and validate all items. Reserve all or none; persist the exact result for retries. Supply holds and expiry.
4 · Verify payment	Request Billing only after full reservation. Accept verified payment only for matching order, invoice, currency and amount.	Keep the unpaid hold active until confirmation, release or authoritative expiry.
5 · Confirm stock	After payment, request Inventory confirmation. Move to confirmed only when payment and reservation validity are verified.	Confirm every original hold atomically or report expired/unavailable. Under Inventory's source policy, remove expiry when confirmed.
6 · Fulfill	Request Delivery acceptance only when eligible. Delivery acceptance makes the order fulfilling.	Keep confirmed holds until the single agreed dispatch/pickup event. Only Inventory deducts stock.
7 · Complete	Mark completed only after verified delivery with the required payment still verified.	Retain fulfilled holds and sale movements; completion does not deduct a second time.

> **Current app limitation**
>
> The demonstration supplies simulated reservation/payment/delivery results. It does not implement Inventory's real stock balance engine, expiry worker or Delivery-triggered deductions.

Rejected reservation

Return a stable rejected result and a per-item reason such as insufficient stock or inactive offer. Sales requests no payment. The current initial Sales UI supports cancelling the order and creating a new draft rather than silently re-reserving an item that already has a final reservation record.

INTEGRATION SPECIFICATION

7. Cancellation, expiry and race handling

Submitted unpaid cancellation

1. Sales enters cancellation_pending, records the reason and blocks fulfillment.
2. Billing authoritatively verifies unpaid cancellation and blocks future charges for that order.
3. Inventory locks the order-control row and records a reservation-blocked tombstone, even if no reservation exists yet.
4. Inventory releases eligible holds, reduces reserved balances, saves the terminal result and commits. No physical movement is recorded for release.
5. Sales marks cancelled only when the unpaid check and no-active-or-future-reservations guarantee are both authoritative.

Race / failure	Required outcome
Release arrives before a delayed reserve	The tombstone makes the later reserve reject. “No holds found right now” is not sufficient release proof.
Payment arrives during cancellation	Keep the verified payment evidence, pause progress and reconcile. Do not silently cancel a paid order or ship without valid stock.
Payment arrives after expiry	Inventory must not revive an expired hold. Sales keeps the order unresolved; agree re-reservation/refund handling outside the initial scope.
Confirm races with expiry	Both lock the same order-control and stock rows. One valid transition wins; the other returns the authoritative current result.
Dispatch races with release	Only the agreed eligible state may win. A fulfilled hold cannot be released; a released/expired hold cannot be fulfilled.
Timeout or lost response	The result is unresolved. Query the original request, or retry the same immutable request reference. Never assume rejection or create a new hold key.

Unpaid expiry

The Inventory group proposes 30 minutes; the duration is not a confirmed class agreement. An Inventory worker expires eligible active holds under locks, reduces only reserved balances, and writes a durable result/notification. Never rely on the customer’s browser timer to change stock.

> **Lifecycle receiver is implemented locally**
>
> Use the signed, versioned POST /api/events/inventory/lifecycle receiver for later expiry, release and fulfillment facts. It validates complete reservation identities, monotonic versions and eligible states. A later lifecycle fact must not overwrite the original reserve operation result. External Inventory delivery remains to be integrated.

INTEGRATION SPECIFICATION

8. Current Sales–Inventory HTTP contract

This page records the **current code's contract**, not a claim that the other group already exposes it. Adapt the actual Inventory service or update both sides under a versioned agreement.

Operation	Current transport
Query request	GET <INVENTORY_URL>/requests/<encoded request_reference>
Submit operation	POST <INVENTORY_URL>/requests/reserve, /confirm or /release
Asynchronous result	POST <Sales base>/api/events/inventory
Identity	Idempotency-Key header = request_reference; result includes matching request_reference and numeric order_id.

Illustrative reserve request

```
{
  "request_reference": "sales:42:inventory:reserve",
  "order_id": 42, "customer_id": 7, "seller_id": 3,
  "currency": "PHP", "product_total": "179.00",
  "items": [{
    "order_item_id": 101, "offer_id": 9, "seller_id": 3,
    "product_name_at_order": "Notebook",
    "quantity": 2, "unit_price_at_order": "89.50"
  }],
  "recipient": {
    "name": "Fictional Recipient", "contact": "DEMO-CONTACT",
    "address": "DEMO-ADDRESS"
  },
  "reservation": null
}
```

The current common envelope includes purchase and recipient fields. Inventory needs references and quantities; do not persist unrelated contact/address/price data as Inventory-owned records. Agree a minimized adapter payload before live use.

Signed messages and retries

Use HTTPS and a distinct Inventory secret. All responses, including 404, and webhook events include `x-sales-timestamp` (Unix milliseconds, within five minutes) and `x-sales-signature` (lowercase HMAC-SHA256 over `timestamp + "." + exact UTF-8 body`). POST uses JSON. GET is signed with an empty body.

Sales queries the original request before sending. An authenticated 404 permits POST of the same payload. `{"status":"pending"}` remains unresolved. The current HTTP timeout is 10 seconds and dispatch lease is 60 seconds. These are Sales transport settings, not reservation lifetimes.

INTEGRATION SPECIFICATION

9. Results, identifiers and compatibility

Current reserve result example

```
{
  "event_reference": "inventory-event-001",
  "request_reference": "sales:42:inventory:reserve",
  "order_id": 42,
  "status": "reserved",
  "expires_at": 1792643400000,
  "reservations": [
    {
      "order_item_id": 101,
      "offer_id": 9,
      "quantity": 2,
      "reservation_id": "res-101"
    }
  ]
}
```

Example expiry: 22 October 2026, 04:30 UTC. Use the actual future expiry at execution time. Reservation IDs are nonblank strings; item/order/offer IDs are numeric in the current wire contract.

Operation	Current accepted result	Important validation
Reserve	reserved + future expires_at + reservations; or rejected	Every item exactly once, correct offer/quantity, distinct reservation IDs. rejected may include short_items.
Confirm	confirmed + explicit NULL expiry (or valid future expiry) + matching reservation identities; or expired	Current code compares the complete item/reservation identity set independent of array order. No new, duplicate or partial holds.
Release	released + no_active_reservations: true	Only return success after release and prevention of future in-flight reservation creation.
Duplicate result	Same event reference and semantically identical content returns a no-op	Conflicting content or a second terminal result is rejected. JSON key order does not create a false conflict.

LOCAL FIXES AND REMAINING EXTERNAL AGREEMENTS

10. Compatibility and policy decisions

Confirmed validity: local alignment complete

Inventory's source design defines confirmed holds as non-expiring, represented by `expires_at = NULL` in the proposed SQL. The Sales confirmation handler, stock guards, simulator and progress display now accept explicit NULL confirmed expiry. Active unpaid holds still require a deadline. Future confirmed expiries remain supported for compatibility with an explicitly expiring provider.

Implemented locally: non-expiring confirmation, semantic event deduplication, order-independent reservation matching, cancellation flag correction and a signed versioned lifecycle receiver. Agree the authoritative validity contract with the real Inventory service and verify it jointly. A separate technical authorization lease may exist only if the groups explicitly design it; it must not be misrepresented as an Inventory hold expiry.

> **No fabricated compatibility**
>
> The simulator now confirms without automatic expiry. Earlier demo records may retain their explicit expiry values; the application preserves those values rather than rewriting historical orders.

Further contract decisions

Contract issue	Required joint decision
Reservation lifecycle	Connect Inventory to the implemented /api/events/inventory/lifecycle v1 receiver. Provide monotonic per-order versions, stable event references and complete matching reservation identities.
Failed payment	Inventory proposes release on failure; the current Sales flow flags reconciliation and uses its cancellation process. Choose one authoritative coordinator and verify Billing eligibility before release.
Validity versus expiry	If a non-expiring confirmed hold is adopted, Sales must distinguish confirmed validity from a technical authorization lease. Document lease semantics separately if introduced.
Dispatch/pickup	Use exactly one agreed event for physical deduction. Delivery acceptance and proof of delivery do not by themselves constitute the chosen stock-deduction signal.

These decisions require code and test changes in the affected services. The Sales adapter has been updated and locally tested. This document does not make the proposed Inventory tables or external services live.

INTEGRATION SPECIFICATION

11. Additive schema and deployment contract

The appendix and companion `System_1_2_3_Inventory_Proposed.sql` extend the original System12 schema. They are a **review proposal**, not an installed migration or proof of MySQL execution.

Design choice	Rationale
No changes to existing group-owned tables	The proposal adds Inventory-owned tables only. Original offer/order/item/history identifiers and accepted snapshots remain intact.
Canonical offer_id	Inventory uses the existing offer/seller composite FK. product_id is derived through seller_offer, preventing redundant seller/product mappings from drifting.
String reservation key	Matches the current Sales response validation without unsafe numeric/string coercion.
One request, many holds	The request ledger deduplicates the whole operation. Each order item has one hold, and the request/item pair identifies a line within the batch.
One event, many movements	The inbox deduplicates a source event. A dispatch event may legitimately produce one sale movement per stock record.
Derived available quantity	The proposed generated column is computed from balances. It is never an independently editable counter.
No automatic data conversion	Existing Inventory data, if any, requires an inspected migration, mapping, backup and reconciliation. Never import over it blindly.

Database permissions

Use a separate privileged migration account. Catalogue's runtime account writes only Catalogue-owned data; Sales reads Catalogue and writes its own records; Inventory reads permitted references and writes only Inventory tables. Do not grant other modules UPDATE access to stock balances or movement history. Enforce append-only movement behavior with grants and controlled write paths.

Conditions before running proposed DDL

1. Confirm shared MySQL versus independent service deployment and actual schema versions.
2. Inspect existing table names, IDs, collations, engine and constraints; back up the target.
3. Review the locally implemented confirmed-validity and lifecycle contract with Inventory, then review the proposed migration.
4. Execute in a disposable database first, run transaction/concurrency tests, then approve rollout.

There are no DROP, TRUNCATE, ALTER, CREATE DATABASE or USE statements in the companion extension. CREATE TABLE still changes a schema and must not be run without the above review.

INTEGRATION SPECIFICATION

12. Processing algorithms and acceptance tests

Reserve transaction

Authenticate → validate request hash/reference → lock order control → reject a cancellation tombstone → lock stock rows in stable order → verify all authoritative order-item mappings and active offers → check all available quantities → create all holds and increment reserved balances → save one immutable batch result and outbox entries → commit. A duplicate key returns the original stored result; a changed payload under the same key is a conflict.

Confirm, release and fulfillment

Confirm: verify Sales' payment authorization under the agreed trust contract, lock the same records, require every original active hold to be eligible, and change all holds together. **Release:** create the order tombstone first, validate release eligibility, release all releasable holds and save the authoritative result. **Fulfill:** deduplicate the agreed Delivery event, verify order/item/stock/reservation links, reduce both balances, mark all holds fulfilled and append sale movements in one transaction.

Acceptance test	Pass condition
Last unit / concurrent requests	Only one order reserves the last available unit; balance never becomes negative.
One short or inactive item	Whole request rejected; no partial hold or counter change. Product/offer IDs identify failures.
Repeated reserve / changed payload	Identical retry returns identical IDs/result; altered body under same key is rejected.
Multi-item request / dispatch	One request can create multiple holds; one dispatch event can create multiple movements without duplicate deductions.
Release before reserve / duplicate release	No later hold can be created; release stays idempotent and counters do not double-decrease.
Confirm vs expiry; release vs fulfill	One legal serialized outcome; no expired/released hold is revived or shipped.
Payment/cancellation race	Paid evidence is retained; Sales remains unresolved until reconciliation.
Stock audit and privilege checks	Every on-hand change has a matching immutable movement; Sales cannot update Inventory.
Existing purchase preservation	Catalogue changes never rewrite submitted Sales names, prices or totals.

These are required joint tests for the proposed integration. Existing Sales tests use SQLite/simulators and do not substitute for MySQL/Inventory transaction tests.

INTEGRATION SPECIFICATION

13. Handoff, open agreements and sources

Decision / deliverable	Responsible groups	Status at issue
Platform, hosting and database access	All groups	Unconfirmed shared deployment; SQLite demo only.
offer_id ↔ seller/product mapping	Catalogue + Inventory + Sales	Proposed canonical offer_id; confirm real data mappings.
Non-expiring confirmed holds	Inventory + Sales	Sales support and regression tests implemented; external service agreement/verification pending.
Unpaid expiry and failed-payment release	Inventory + Sales + Billing	30 minutes proposed; agree coordinator and exact release conditions.
Dispatch versus pickup deduction	Inventory + Delivery + Sales	Choose one authoritative event; no double deduction.
Reservation lifecycle notification/query	Inventory + Sales	Signed lifecycle receiver v1 implemented; Inventory must provide versioned authenticated events.
Cancellation tombstone guarantees	Inventory + Sales + Billing	Required to prove no active/future holds and no successful payment.
Existing data migration and privileges	Database owner + affected groups	No migration, grants or live integration verified by this document.

Implementation handoff

Inventory group: review the schema; implement atomic stock processing, durable request/event results, expiry and release guarantees; publish its actual authenticated interface. **Sales group:** verify the implemented confirmed-validity model and versioned lifecycle receiver against real Inventory; keep immutable purchases, backend guards and reconciliation. **Joint review:** agree contract version, run the acceptance matrix on real service/database instances and sign off before enabling production traffic.

> **Recommended review sequence**
>
> Inventory first reviews identities and invariants; Sales reviews the implemented contract and confirmed-validity model; Delivery confirms the deduction event; Billing confirms cancellation guarantees. Only then should database and service acceptance testing begin.

TRACEABILITY AND APPROVAL STATUS

14. Source basis and review record

Reference basis

1. *System 1 and 2 Document*, packaged 11-page PDF: scope/schema (pp. 1–7), Sales/integration rules (pp. 8–10), open agreements (pp. 10–11). Matching downloaded PDF SHA-256 begins `29fde37e26a486a3`.
2. *System12.sql*: original eight Catalogue/Sales tables, composite seller FKs, generated monetary line_total and v_orderable_offer. SHA-256 begins `31e7c6720a07fa2d`.
3. *Inventory Management Deliverables*, packaged 21-page PDF: scope/ownership (pp. 2–5), entities (pp. 8–12), invariants/states (pp. 13–17), integration/workflow (pp. 18–21).
4. Current CampusERP: `src/sales.mjs`, `src/integrations.mjs`, `src/db.mjs`, `docs/INTEGRATION.md` and `docs/DECISIONS.md`.

> **Review status**
>
> The Sales-side fixes are regression- and browser-tested, and the PDF layout is checked. The SQL has not been executed against MySQL, no Inventory service has been implemented by this document, and no class-wide contract is claimed.

Sales representative: _____ Inventory representative: _____

Database / integration reviewer: _____ Approval date: _____

Review checklist

- Offer, order, item and reservation references agreed across all affected systems.
- Shared-database or independent-API deployment chosen explicitly.
- Expiry, confirmation validity, cancellation and stock-deduction policies approved.
- Proposed DDL tested in a disposable database and migration reviewed.
- Authentication, idempotency, concurrency and reconciliation acceptance tests passed.
- Actual integration outcomes recorded separately from simulator demonstrations.

PROPOSED ADDITIVE SQL · NOT A LIVE MIGRATION

Appendix A.1. Proposed MySQL extension

Companion file: `System_1_2_3_Inventory_Proposed.sql` · Lines 1–48. Read sections 2, 10 and 11 before use. DDL is a proposal; it has not been executed.

```
-- PROPOSED integration extension for System 3, version 0.2, 2026-10-22.
-- Review draft, NOT an executed or approved production migration.
-- Target: MySQL 8.0.16+; InnoDB; UTC DATETIME values.
-- Preconditions: original System12.sql tables exist in the selected dedicated DB.
-- No CREATE DATABASE, USE, DROP, TRUNCATE, ALTER, or sample data.
-- STOP if any of these proposed names already exist; reconcile first.
-- Cross-module FKs apply only to an AGREED shared MySQL deployment.
-- API-separated deployments need verified reference mappings instead.

CREATE TABLE inventory_order_control (
  order_id INT NOT NULL,
  reservation_blocked TINYINT NOT NULL DEFAULT 0,
  lifecycle_version BIGINT NOT NULL DEFAULT 0,
  block_reason VARCHAR(500),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
    ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (order_id),
  CONSTRAINT fk_inv_control_order FOREIGN KEY (order_id)
    REFERENCES sales_order (order_id),
  CONSTRAINT chk_inv_block CHECK (reservation_blocked IN (0,1)),
  CONSTRAINT chk_inv_life_version CHECK (lifecycle_version >= 0)
) ENGINE=InnoDB;

CREATE TABLE inventory_request (
  request_reference VARCHAR(100) NOT NULL,
  order_id INT NOT NULL,
  operation VARCHAR(20) NOT NULL,
  payload_sha256 CHAR(64) NOT NULL,
  result_status VARCHAR(20) NOT NULL,
  response_json JSON,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (request_reference),
  KEY ix_inv_request_order (order_id),
  CONSTRAINT fk_inv_request_order FOREIGN KEY (order_id)
    REFERENCES sales_order (order_id),
  CONSTRAINT chk_inv_operation CHECK
    (operation IN ('reserve','confirm','release')),
  CONSTRAINT chk_inv_result CHECK
    (result_status IN ('pending','reserved','rejected',
      'confirmed','expired','released'))
) ENGINE=InnoDB;

CREATE TABLE inventory (
  inventory_id INT NOT NULL AUTO_INCREMENT,
  offer_id INT NOT NULL,
  seller_id INT NOT NULL,
  quantity_on_hand INT NOT NULL DEFAULT 0,
  quantity_reserved INT NOT NULL DEFAULT 0,
```

PROPOSED ADDITIVE SQL · NOT A LIVE MIGRATION

Appendix A.2. Proposed MySQL extension

Companion file: `System_1_2_3_Inventory_Proposed.sql` · Lines 49–96. Read sections 2, 10 and 11 before use. DDL is a proposal; it has not been executed.

```
  available_quantity INT GENERATED ALWAYS AS
    (quantity_on_hand - quantity_reserved) STORED,
  reorder_level INT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
    ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (inventory_id),
  UNIQUE KEY uq_inv_offer (offer_id),
  UNIQUE KEY uq_inv_identity (inventory_id, offer_id, seller_id),
  CONSTRAINT fk_inv_offer FOREIGN KEY (offer_id, seller_id)
    REFERENCES seller_offer (offer_id, seller_id),
  CONSTRAINT chk_inv_balance CHECK
    (quantity_on_hand >= 0 AND quantity_reserved >= 0
     AND quantity_reserved <= quantity_on_hand),
  CONSTRAINT chk_inv_reorder CHECK (reorder_level >= 0)
) ENGINE=InnoDB;

CREATE TABLE stock_reservation (
  reservation_id VARCHAR(64) NOT NULL,
  request_reference VARCHAR(100) NOT NULL,
  inventory_id INT NOT NULL,
  offer_id INT NOT NULL,
  seller_id INT NOT NULL,
  order_id INT NOT NULL,
  order_item_id INT NOT NULL,
  quantity INT NOT NULL,
  reservation_status VARCHAR(20) NOT NULL,
  expires_at DATETIME(3),
  release_reason VARCHAR(500),
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
    ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (reservation_id),
  UNIQUE KEY uq_res_item (order_item_id),
  UNIQUE KEY uq_res_request_item (request_reference, order_item_id),
  KEY ix_res_order (order_id),
  KEY ix_res_expiry (reservation_status, expires_at),
  CONSTRAINT fk_res_request FOREIGN KEY (request_reference)
    REFERENCES inventory_request (request_reference),
  CONSTRAINT fk_res_inventory FOREIGN KEY
    (inventory_id, offer_id, seller_id)
    REFERENCES inventory (inventory_id, offer_id, seller_id),
  CONSTRAINT fk_res_order FOREIGN KEY (order_id)
    REFERENCES sales_order (order_id),
  CONSTRAINT fk_res_item FOREIGN KEY (order_item_id)
    REFERENCES order_item (order_item_id),
  CONSTRAINT chk_res_quantity CHECK (quantity > 0),
  CONSTRAINT chk_res_status CHECK (reservation_status IN
```

PROPOSED ADDITIVE SQL · NOT A LIVE MIGRATION

Appendix A.3. Proposed MySQL extension

Companion file: `System_1_2_3_Inventory_Proposed.sql` · Lines 97–144. Read sections 2, 10 and 11 before use. DDL is a proposal; it has not been executed.

```
    ('active','confirmed','fulfilled','released','expired')),
  CONSTRAINT chk_res_expiry CHECK
    ((reservation_status <> 'active' OR expires_at IS NOT NULL)
     AND (reservation_status <> 'confirmed' OR expires_at IS NULL)),
  CONSTRAINT chk_res_reason CHECK
    (reservation_status <> 'released' OR
     (release_reason IS NOT NULL AND LENGTH(TRIM(release_reason)) > 0))
) ENGINE=InnoDB;

CREATE TABLE inventory_event_inbox (
  source_event_reference VARCHAR(100) NOT NULL,
  source_system VARCHAR(30) NOT NULL,
  event_type VARCHAR(30) NOT NULL,
  payload_sha256 CHAR(64) NOT NULL,
  result_json JSON,
  processed_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (source_event_reference)
) ENGINE=InnoDB;

CREATE TABLE stock_movement (
  movement_id BIGINT NOT NULL AUTO_INCREMENT,
  inventory_id INT NOT NULL,
  reservation_id VARCHAR(64),
  order_id INT,
  source_event_reference VARCHAR(100) NOT NULL,
  movement_type VARCHAR(20) NOT NULL,
  quantity INT NOT NULL,
  on_hand_after INT NOT NULL,
  performed_by VARCHAR(100) NOT NULL,
  remarks VARCHAR(500),
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (movement_id),
  UNIQUE KEY uq_move_reservation (reservation_id),
  UNIQUE KEY uq_move_event_stock
    (source_event_reference, inventory_id, movement_type),
  CONSTRAINT fk_move_inventory FOREIGN KEY (inventory_id)
    REFERENCES inventory (inventory_id),
  CONSTRAINT fk_move_reservation FOREIGN KEY (reservation_id)
    REFERENCES stock_reservation (reservation_id),
  CONSTRAINT fk_move_order FOREIGN KEY (order_id)
    REFERENCES sales_order (order_id),
  CONSTRAINT fk_move_event FOREIGN KEY (source_event_reference)
    REFERENCES inventory_event_inbox (source_event_reference),
  CONSTRAINT chk_move_type CHECK (movement_type IN
    ('restock','sale','return','adjustment_in','adjustment_out')),
  CONSTRAINT chk_move_quantity CHECK (quantity > 0),
  CONSTRAINT chk_move_balance CHECK (on_hand_after >= 0),
  CONSTRAINT chk_move_reservation CHECK
```

PROPOSED ADDITIVE SQL · NOT A LIVE MIGRATION

Appendix A.4. Proposed MySQL extension

Companion file: `System_1_2_3_Inventory_Proposed.sql` · Lines 145–190. Read sections 2, 10 and 11 before use. DDL is a proposal; it has not been executed.

```
    ((movement_type = 'sale' AND reservation_id IS NOT NULL
      AND order_id IS NOT NULL) OR
     (movement_type <> 'sale' AND reservation_id IS NULL)),
  CONSTRAINT chk_move_reason CHECK
    (movement_type NOT IN ('adjustment_in','adjustment_out') OR
     (remarks IS NOT NULL AND LENGTH(TRIM(remarks)) > 0))
) ENGINE=InnoDB;

CREATE TABLE inventory_event_outbox (
  event_reference VARCHAR(100) NOT NULL,
  order_id INT NOT NULL,
  event_type VARCHAR(30) NOT NULL,
  payload_json JSON NOT NULL,
  lifecycle_version BIGINT,
  delivery_status VARCHAR(20) NOT NULL DEFAULT 'pending',
  attempts INT NOT NULL DEFAULT 0,
  created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (event_reference),
  UNIQUE KEY uq_inv_lifecycle_version (order_id, lifecycle_version),
  KEY ix_inv_outbox_delivery (delivery_status, created_at),
  CONSTRAINT fk_inv_outbox_order FOREIGN KEY (order_id)
    REFERENCES sales_order (order_id),
  CONSTRAINT chk_inv_delivery CHECK
    (delivery_status IN ('pending','delivered')),
  CONSTRAINT chk_inv_attempts CHECK (attempts >= 0),
  CONSTRAINT chk_inv_outbox_version CHECK
    (event_type <> 'reservation.lifecycle' OR
     (lifecycle_version IS NOT NULL AND lifecycle_version > 0))
) ENGINE=InnoDB;

-- APPLICATION REQUIREMENTS: FKs alone do not enforce these.
-- Validate each order_item belongs to the supplied order, offer, and seller;
-- quantity equals accepted Sales quantity; request operation/order matches.
-- Validate sale movement matches reservation's inventory/order/quantity.
-- Lock order-control FIRST, then inventory records by ascending inventory_id,
-- then reservations; every mutating operation follows this same lock order.
-- Maintain reserved == SUM(active + confirmed reservation quantities).
-- No stock write outside the balance+movement/reservation transaction.
-- No network I/O while a stock transaction is open.
-- Inventory service alone writes these tables; Sales/Catalogue do not.
-- Enforce immutable movements using runtime grants and controlled procedures.
-- Confirmation expiry follows Inventory's proposal (NULL after confirmed).
-- Sales now supports NULL confirmed expiry and signed lifecycle v1 updates.

-- Increment control.lifecycle_version under the order-control lock and copy it
-- into the lifecycle outbox row/payload in the SAME Inventory transaction.
```